AXY CRYSTAL

MERGE EDİTİON

GODOT 4 İLE SIFIRDAN YAPIM REHBERİ

Sıfır Godot Bilgisiyle Profesyonel Mobil Oyun Geliştirme

14 Bölüm • Adım Adım Kodlar • Tam Kaynak Kodu

📖 ÖNSÖZ — Bu Kitap Hakkında

Bu kitabı kim için yazdım?

Bu rehber, Godot motorunu hiç kullanmamış, belki oyun geliştirmeye yeni adım atmış kişiler için hazırlandı. Tek bir satır kod bilmeden başlayıp, sonunda tam çalışan bir mobil puzzle oyunu yapabileceksiniz.

Ne öğreneceğim?

- Godot 4 arayüzünü tanımak ve proje kurmak
- GDScript ile temel programlama
- 5×7 grid (ızgara) sistemi oluşturmak
- Dokunmatik kontrol (hold ve bırakma mekaniği)
- BFS (genişlik öncelikli arama) algoritması ile bağlı taşları bulmak
- Taşların birleşmesi (merge) ve animasyonlar
- Yerçekimi sistemi — taşların aşağı düşmesi
- Yeni taş üretme (spawn) sistemi
- Puan ve coin (para) sistemi
- Power-up (güçlendirme) sistemi: Bomba, Smash, Dondur, vb.
- Ses efektleri
- Oyun kayıt/yükleme sistemi

Nasıl kullanmalıyım?

Her bölümü sırasıyla okuyun. Kodları birebir yazın veya kopyalayın. "DİKKAT" kutularına özellikle dikkat edin — sık yapılan hatalar orada açıklanır. "İPUCU" kutularındaki bilgiler ilerleride işinize yarayacak kısayollardır.

> ⚠️ **DİKKAT:** Godot 4.x kullanıyoruz. Godot 3.x ile bu rehberdeki bazı kodlar çalışmaz. İndirirken mutlaka 4.x seçin.

📋 İÇİNDEKİLER

Bölüm 1 Godot Nedir? Kurulum ve Arayüz 3
Bölüm 2 GDScript'e Hızlı Giriş — Programlamanın ABC'si 5
Bölüm 3 Proje Kurulumu ve Klasör Yapısı 8
Bölüm 4 Grid Sistemi — 5×7 Izgara Oluşturmak 11
Bölüm 5 Crystal (Taş) Sahnesi — Her Renk Bir Değer 14
Bölüm 6 Hold Mekaniği — Basılı Tut ve Eşleşenleri Bul (BFS) 18
Bölüm 7 Merge Sistemi — Taşları Birleştir 23
Bölüm 8 Yerçekimi — Taşlar Düşsün 27
Bölüm 9 Spawn Sistemi — Yeni Taşlar Üret 30
Bölüm 10 Puan, Coin ve HUD Arayüzü 34
Bölüm 11 Power-up Sistemi — Özel Güçler 38
Bölüm 12 Ses Efektleri 44
Bölüm 13 Kayıt ve Yükleme Sistemi 47
Bölüm 14 Animasyonlar, Son Dokunuşlar ve Yayınlama 50

BÖLÜM 1

Godot Nedir?

Kurulum ve Arayüze İlk Bakış

1.1 Godot Nedir?

Godot, tamamen ücretsiz ve açık kaynaklı bir oyun motorudur. Unity veya Unreal Engine gibi büyük motorların aksine lisans ücreti yoktur. 2D ve 3D oyunlar yapabilir, masaüstü, mobil ve web platformlarına yayınlayabilirsiniz.

> 💡 **İPUCU:** Godot'u indirmek için: https://godotengine.org/download — "Godot 4" yazan sürümü seçin. Mono/.NET gerekmez, sadece standart sürümü indirin.

1.2 Kurulum

Adım 1: Godot websitesine gidin ve işletim sisteminize uygun sürümü indirin.

Adım 2: İndirilen ZIP dosyasını açın. Kurulum gerektirmez — sadece .exe (Windows) veya uygulamayı çalıştırın.

Adım 3: Godot Project Manager açılır. Buradan yeni proje oluşturacağız.

1.3 Arayüzün Bölümleri

Godot'u açtığınızda birkaç farklı panel görürsünüz. Her birinin ne işe yaradığını öğrenmek çok önemlidir:

Panel Adı	Ne İşe Yarar?
Scene (Sahne)	Oyunun yapısı burada görünür. Her sahne, birbirine bağlı "node" (düğüm) denilen nesnelerden oluşur.
FileSystem	Sol altta — projenizin tüm dosyaları burada listelenir. Yeni dosya oluşturmak için sağ tıklayın.
Inspector	Sağ tarafta — seçili node'un özelliklerini (renk, boyut, konum vb.) buradan değiştirirsiniz.
Viewport	Ortada — oyununuzu görsel olarak buraya yerleştirirsiniz. Tıklayıp sürükleyerek nesneleri taşıyabilirsiniz.
Script Editor	Kod yazmak için — bir node'a çift tıklayıp "Attach Script" derseniz kod editörü açılır.
Output	Alt kısımda — print() ile yazdırdığınız mesajlar ve hatalar burada görünür. Debugging'de çok kullanılır.

1.4 Node (Düğüm) Kavramı — Godot'un Temeli

Godot'ta her şey bir "Node" dur. Bir sahne (scene), birbirine bağlı node'lardan oluşur. Örneğin:

- **Node2D** — 2D dünyada konum tutan temel node
- **Sprite2D** — Resim gösteren node
- **Label** — Yazı gösteren node
- **Button** — Tıklanabilir buton
- **AudioStreamPlayer** — Ses çalan node

> 📌 **NOT:** Node'lar bir ağaç yapısı oluşturur. Üst node'u (parent) taşıdığınızda alt node'lar (child) da onunla birlikte hareket eder. Bu hiyerarşiyi anlamak çok önemlidir.

1.5 Sahne (Scene) Kavramı

Bir sahne, birden fazla node'u bir araya getiren dosyadır (.tscn uzantılı). Oyunumuzda şu sahneler olacak:

- **Main.tscn** — Ana oyun sahnesi (grid, HUD, tüm mantık)
- **Crystal.tscn** — Tek bir taş (birden fazla kez kullanılacak)
- **GameOver.tscn** — Oyun bitti ekranı

> 💡 **İPUCU:** "Prefab" yerine Godot'ta "Scene" diyoruz. Bir scene'i birden fazla kez "instance" edebilirsiniz — tıpkı bir kalıptan birden fazla nesne dökmek gibi.

BÖLÜM 2

GDScript'e Hızlı Giriş

Programlamanın ABC'si

2.1 GDScript Nedir?

GDScript, Godot'un kendi programlama dilidir. Python'a çok benzer — girintileme (indentation) ile bloklar oluşturulur. Noktalı virgül gerekmez. Öğrenmesi kolaydır.

2.2 Temel Sözdizimi

Değişkenler (Variables)

Değişken, bir veri saklamak için kullanılan kutudur. GDScript'te iki tür değişken tanımı vardır:

```
# var — değeri değişebilir
var puan = 0
var oyuncu_adi = "Ahmet"
var oyun_devam = true

# const — değeri hiç değişmez (sabit)
const SUTUN_SAYISI = 5
const SATIR_SAYISI = 7

# Tip belirterek tanımlama (önerilen)
var puan: int = 0
var isim: String = "Oyuncu"
var hiz: float = 1.5
var aktif: bool = true
```

Koşullar (if/elif/else)

```
var skor = 100

if skor >= 1000:
    print("Harika!")      # 4 boşluk veya 1 tab ile girintiledik
elif skor >= 500:
    print("İyi!")
else:
    print("Daha çok çalış!")
```

Döngüler (Loops)

```
# for döngüsü — belirli sayıda tekrar
for i in range(5):         # 0,1,2,3,4
    print(i)

# Dizi üzerinde döngü
var meyveler = ["elma", "armut", "kiraz"]
```

```
for meyve in meyveler:
    print(meyve)

# while döngüsü — koşul doğru olduğu sürece
var sayac = 0
while sayac < 3:
    print(sayac)
    sayac += 1
```

Fonksiyonlar (Functions)

```
# func anahtar kelimesi ile fonksiyon tanımla
func selam_ver():
    print("Merhaba!")

# Parametre alan fonksiyon
func toplama(a: int, b: int) -> int:
    return a + b

# Kullanım:
selam_ver()           # Çağırma
var sonuc = toplama(3, 5)    # sonuc = 8
```

Diziler (Arrays) ve Sözlükler (Dictionaries)

```
# Array — sıralı liste
var liste = [10, 20, 30]
liste.append(40)        # Sona ekle
print(liste[0])         # İlk eleman: 10
print(liste.size())     # Eleman sayısı: 4

# Dictionary — anahtar:değer çifti
var oyuncu = {
    "isim": "Ahmet",
    "puan": 1500,
    "seviye": 3
}
print(oyuncu["isim"])   # Ahmet
oyuncu["puan"] = 2000   # Değeri değiştir
```

2.3 Godot'a Özel Kavramlar

_ready() ve _process()

```
extends Node2D    # Bu script hangi node tipine ait?

# Sahne yüklendiğinde BİR KERE çalışır
func _ready():
    print("Sahne hazır!")
    puan = 0

# Her karede (frame) sürekli çalışır (~60 kez/saniye)
# delta = önceki kare ile arası geçen süre (saniye)
func _process(delta: float):
    pass    # Şimdilik boş
```

> 💡 **İPUCU:** _process() her karede çalışır. Ağır işlemleri buraya koymayın. Sadece sürekli kontrol edilmesi gereken şeyler için kullanın.

Signal (Sinyal) — Haberleşme Sistemi

Sinyaller, node'lar arasında haberleşmek için kullanılır. Örneğin bir buton tıklandığında "pressed" sinyali yayar:

```
# Sinyal tanımla
signal merge_tamamlandi(yeni_deger)

# Sinyal yay
emit_signal("merge_tamamlandi", 128)

# Sinyali dinle (başka bir scriptten)
func _ready():
    kristal.connect("merge_tamamlandi", _merge_oldu)

func _merge_oldu(deger):
    print("Merge sonucu: ", deger)
```

BÖLÜM 3

Proje Kurulumu

Klasör Yapısı ve İlk Ayarlar

3.1 Yeni Proje Oluşturma

Adım 1: Godot'u açın → Project Manager gelir → "New Project" butonuna tıklayın.
Adım 2: Proje adı: "AxycrystalMerge" yazın. Boşluk kullanmayın.
Adım 3: Renderer (render motoru) seçin: "Forward+" (mobil için iyi çalışır).
Adım 4: "Create & Edit" butonuna tıklayın. Proje açılır.

3.2 Proje Ayarları (Project Settings)

Üst menüden Project → Project Settings açın. Şu ayarları yapın:

Ayar Yolu	Değer
Display → Window → Size → Width	480 (mobil genişlik)
Display → Window → Size → Height	900 (mobil yükseklik)
Display → Window → Stretch → Mode	"canvas_items" seçin
Display → Window → Stretch → Aspect	"expand" seçin
Rendering → 2D → Snap → Snap 2D Transforms	Açık (ON) — piksel kaymasını önler
Application → Config → Name	"AXY CRYSTAL" yazın

> 💡 **İPUCU:** "Stretch Mode: canvas_items + Aspect: expand" kombinasyonu farklı ekran boyutlarında oyununun düzgün ölçeklenmesini sağlar. Mobil geliştirmede olmazmazıdır.

3.3 Klasör Yapısı

FileSystem panelinde şu klasörleri oluşturun (sağ tık → New Folder):

```
res://                    ← Projenin kök dizini
```

```
├── scenes/                ← Tüm .tscn sahneleri
│   ├── Main.tscn          ← Ana oyun sahnesi
│   ├── Crystal.tscn       ← Tek taş sahnesi
│   └── GameOver.tscn      ← Oyun bitti
├── scripts/               ← Tüm .gd script dosyaları
│   ├── GameManager.gd     ← Ana oyun mantığı
│   ├── Crystal.gd         ← Taş davranışı
│   └── HUD.gd             ← Arayüz mantığı
├── audio/                 ← Ses dosyaları (.wav, .ogg)
└── fonts/                 ← Font dosyaları (.ttf)
```

⚠️ **DİKKAT:** Dosya ve klasör adlarında Türkçe karakter ve boşluk kullanmayın. "taş_rengi" yerine "tas_rengi" yazın. Bu, özellikle yayınlama aşamasında sorun çıkarır.

3.4 Ana Sahneyi (Main.tscn) Oluşturma

Adım 1: Scene panelinde "+ New Scene" tıklayın.

Adım 2: "Other Node" seçin → "Node2D" arayın → Oluştur.

Adım 3: Node adını "Main" olarak değiştirin (sağ tık → Rename).

Adım 4: Ctrl+S ile kaydedin → scenes/Main.tscn olarak adlandırın.

Adım 5: Project Settings → Application → Run → Main Scene → "scenes/Main.tscn" seçin.

✅ **TAMAMLANDI:** Şu ana kadar: Godot kurulumu, proje ayarları ve klasör yapısı tamam. Artık gerçek oyun içeriğini oluşturmaya başlayabiliriz.

BÖLÜM 4

Grid Sistemi

5×7 Izgara Oluşturmak

4.1 Grid Nedir ve Neden Önemlidir?

AXY Crystal'de oyun alanı 5 sütun × 7 satır = 35 hücreden oluşan bir ızgaradır. Her hücre bir taş içerebilir ya da boş olabilir. Oyunun tüm mantığı bu ızgara üzerine kuruludur.

Grid'i iki şekilde düşünebiliriz:

- **G matrisi (mantıksal):** Bellekte tutulan 2 boyutlu dizi. G[satır][sütun] = taşın değeri. 0 = boş hücre.
- **Görsel grid:** Ekranda gördüğünüz renkli kutular.

> 💡 **İPUCU:** Her zaman önce G matrisini güncelleyin, sonra ekranı güncelleyin. Asla tersi olmaz. Bu kural kodun tutarlı kalmasını sağlar.

4.2 Main.tscn Node Yapısı

Main sahnesinde şu node'ları oluşturun:

```
Main (Node2D)
├── Background (ColorRect)          ← Koyu mavi arkaplan
├── GridContainer (Node2D)          ← Taşları bu node altına ekleyeceğiz
├── HUD (CanvasLayer)               ← Puan, coin, hedef göstergeleri
│   ├── ScoreLabel (Label)
│   ├── CoinLabel (Label)
│   ├── TargetLabel (Label)
│   └── PowerupPanel (HBoxContainer)
└── GameOverScreen (Control)        ← Başlangıçta gizli
```

Background oluşturma:

- Scene panelinde Main'e sağ tık → "Add Child Node"
- "ColorRect" arayın ve ekleyin
- Inspector'da Color: #0A0E24 (koyu lacivert) yapın
- Layout → Full Rect seçin (tüm ekranı kaplar)

4.3 GameManager.gd — Ana Script

Main node'una sağ tık → "Attach Script" → scripts/GameManager.gd olarak kaydedin. İçeriği şu şekilde doldurun:

```
extends Node2D

# ── SABİTLER ──────────────────────────────────────────
```

```
const COLS: int = 5          # Sütun sayısı
const ROWS: int = 7          # Satır sayısı
const CELL_SIZE: float = 62.0  # Her hücrenin piksel boyutu
const CELL_GAP: float = 6.0    # Hücreler arası boşluk

# Hedef değerleri dizisi (20 bölüm)
const TARGETS: Array = [
    2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144,
    524288, 1048576, 2097152, 4194304, 8388608, 16777216,
    33554432, 67108864, 134217728, 268435456, 536870912, 1073741824
]

# Power-up maliyetleri
const PU_COST: Dictionary = {
    "swap": 40,
    "undo": 50,
    "smash": 50,
    "shuffle": 80,
    "bomb": 120,
    "freeze": 60
}

# ── DEĞİŞKENLER ────────────────────────────────────────
var G: Array = []        # 2D grid matrisi: G[r][c] = deger (0=bos)
var score: int = 0       # Toplam puan
var coins: int = 30      # Başlangıç coini
var target_idx: int = 0  # Aktif hedef indeksi
var is_active: bool = false  # Oyun aktif mi?
var is_busy: bool = false    # Animasyon sürüyor mu?

# Crystal node referansları: key = "satir-sutun", value = Crystal node
var block_elements: Dictionary = {}

# Hold mekaniği
var hold_cell: Vector2i = Vector2i(-1, -1)  # Basılan hücre
var hold_matches: Array = []  # BFS ile bulunan eşleşen hücreler
var hold_timer: float = 0.0   # Basılı tutma süresi
const HOLD_THRESHOLD: float = 0.175  # 175ms eşiği
var is_holding: bool = false

# Diğer
var history: Array = []   # Geri alma geçmişi
var freeze_active: bool = false  # Dondur power-up aktif mi?
var last_merge_count: int = 0    # Son merge taş sayısı

# Node referansları
@onready var grid_container = $GridContainer

func _ready():
    init_grid()
    is_active = true

# ── GRİD BAŞLATMA ──────────────────────────────────────
func init_grid():
    G = []
    for r in range(ROWS):
        var row = []
        for c in range(COLS):
            row.append(0)
        G.append(row)
    spawn_tiles()   # İlk taşları doldur
```

4.4 Hücre Konumunu Hesaplama

Her taşın piksel koordinatını hesaplayan fonksiyon — taşları ekranda doğru yere yerleştirmek için kullanılır:

```
# Satır ve sütun indeksinden piksel konumu hesapla
func cell_to_pos(row: int, col: int) -> Vector2:
    var x = col * (CELL_SIZE + CELL_GAP)
    var y = row * (CELL_SIZE + CELL_GAP)
    return Vector2(x, y)

# Piksel konumundan satır/sütun hesapla (ters işlem)
func pos_to_cell(pos: Vector2) -> Vector2i:
    var col = int(pos.x / (CELL_SIZE + CELL_GAP))
    var row = int(pos.y / (CELL_SIZE + CELL_GAP))
    return Vector2i(row, col)

# Geçerli hücre mi? (sınırlar içinde mi?)
func is_valid_cell(row: int, col: int) -> bool:
    return row >= 0 and row < ROWS and col >= 0 and col < COLS
```

4.5 GridContainer'ı Ortalama

Grid'in ekranın ortasında durması için GridContainer'ın konumunu hesaplamanız gerekir. Main._ready() içine ekleyin:

```
func _ready():
    # Grid toplam boyutunu hesapla
    var grid_w = COLS * CELL_SIZE + (COLS - 1) * CELL_GAP
    var grid_h = ROWS * CELL_SIZE + (ROWS - 1) * CELL_GAP

    # Ekran boyutunu al
    var screen_w = get_viewport().get_visible_rect().size.x
    var screen_h = get_viewport().get_visible_rect().size.y

    # Grid'i ortala (HUD için 120px üstte bırak, 80px altta)
    var offset_x = (screen_w - grid_w) / 2.0
    var offset_y = 120.0 + (screen_h - 120 - 80 - grid_h) / 2.0
    grid_container.position = Vector2(offset_x, offset_y)

    init_grid()
    is_active = true
```

BÖLÜM 5

Crystal (Taş) Sahnesi

Her Renk Bir Değer

5.1 Crystal.tscn Sahnesini Oluşturma

Adım 1: Scene → New Scene → "Other Node" → "Panel" seçin. Adını "Crystal" yapın.
Adım 2: scenes/Crystal.tscn olarak kaydedin.
Adım 3: Crystal node'una bir Label ekleyin (çocuk node). Adı "NumberLabel" olsun.
Adım 4: Crystal'a bir script ekleyin: scripts/Crystal.gd

Crystal node yapısı:

```
Crystal (Panel)
└── NumberLabel (Label)    ← Taşın sayısını gösterir
```

5.2 Taş Boyutu ve Görünümü

Crystal (Panel) seçiliyken Inspector'dan:

- Custom Minimum Size: 62 × 62 px
- Size: 62 × 62 px

NumberLabel seçiliyken Inspector'dan:

- Layout: Full Rect (taşın tamamını kapla)
- Horizontal Alignment: Center
- Vertical Alignment: Center
- Autowrap Mode: Off

5.3 Renk Sistemi

Her taş değeri farklı bir renk alır. Crystal.gd scriptini şu şekilde yazın:

```
extends Panel

var cell_value: int = 2   # Bu taşın değeri
var grid_row: int = 0
var grid_col: int = 0

@onready var number_label: Label = $NumberLabel

# Değer → renk eşleşme tablosu
const VALUE_COLORS: Dictionary = {
    2:       Color("#be185d"),   # Pembe
    4:       Color("#15803d"),   # Yeşil
    8:       Color("#1e40af"),   # Mavi
    16:      Color("#c2410c"),   # Turuncu
```

```
    32:      Color("#6d28d9"),   # Mor
    64:      Color("#0e7490"),   # Cyan
    128:     Color("#a16207"),   # Sarı-kahve
    256:     Color("#991b1b"),   # Kırmızı
    512:     Color("#065f46"),   # Koyu yeşil
    1024:    Color("#78350f"),   # Kahve
    2048:    Color("#fbbf24"),   # Altın
    4096:    Color("#ff0080"),   # Neon pembe
    8192:    Color("#00ffcc"),   # Neon cyan
}

# Taşı başlat: değer ve konum ver
func setup(value: int, row: int, col: int):
    cell_value = value
    grid_row = row
    grid_col = col
    update_visuals()

# Görsel güncelle (renk + yazı)
func update_visuals():
    number_label.text = format_value(cell_value)
    number_label.add_theme_font_size_override("font_size", calc_font_size())

    var color = get_color_for_value(cell_value)
    add_theme_stylebox_override("panel", create_stylebox(color))

# Değeri kısa formata çevir (1048576 → "1M")
func format_value(v: int) -> String:
    if v >= 1073741824: return str(v / 1073741824) + "G"
    if v >= 1048576:    return str(v / 1048576) + "M"
    if v >= 1024:       return str(v / 1024) + "K"
    return str(v)

# Font boyutunu değere göre ayarla
func calc_font_size() -> int:
    var s = format_value(cell_value)
    if s.length() >= 6: return 11
    if s.length() >= 4: return 14
    if s.length() >= 3: return 18
    return 24

# Değere uygun rengi bul
func get_color_for_value(v: int) -> Color:
    if VALUE_COLORS.has(v):
        return VALUE_COLORS[v]
    # Büyük değerler için döngüsel renk
    var keys = VALUE_COLORS.keys()
    var idx = int(log(v) / log(2)) % keys.size()
    return VALUE_COLORS[keys[idx]]

# StyleBox oluştur (yuvarlak köşeli, renkli)
func create_stylebox(color: Color) -> StyleBoxFlat:
    var style = StyleBoxFlat.new()
    style.bg_color = color
    style.corner_radius_top_left = 12
    style.corner_radius_top_right = 12
    style.corner_radius_bottom_left = 12
    style.corner_radius_bottom_right = 12
    style.border_width_left = 2
    style.border_width_right = 2
    style.border_width_top = 2
    style.border_width_bottom = 2
    style.border_color = color.lightened(0.3)
    style.shadow_color = Color(color.r, color.g, color.b, 0.5)
    style.shadow_size = 6
    style.shadow_offset = Vector2(0, 3)
    return style
```

5.4 Taş Oluşturma Fonksiyonu (GameManager'da)

GameManager.gd'e eklenecek — her taş için bir Crystal instance oluşturur:

```
# Crystal sahnesi yolu
const CRYSTAL_SCENE = preload("res://scenes/Crystal.tscn")

# Yeni bir taş oluştur ve grid'e ekle
func create_block(row: int, col: int, value: int) -> Node:
    # Aynı hücrede eski taş varsa sil
    var key = "%d-%d" % [row, col]
    if block_elements.has(key):
        block_elements[key].queue_free()
        block_elements.erase(key)

    # Yeni Crystal instance oluştur
    var crystal = CRYSTAL_SCENE.instantiate()
    grid_container.add_child(crystal)

    # Konumu ayarla
    crystal.position = cell_to_pos(row, col)
    crystal.setup(value, row, col)

    # Referansı kaydet
    block_elements[key] = crystal

    return crystal

# Belirli hücredeki taşı sil
func remove_block(row: int, col: int):
    var key = "%d-%d" % [row, col]
    if block_elements.has(key):
        block_elements[key].queue_free()
        block_elements.erase(key)
```

BÖLÜM 6

Hold Mekaniği

Basılı Tut ve Eşleşenleri Bul (BFS)

6.1 Hold Mekaniğini Anlamak

Oyunun temel mekaniği şudur: Bir taşa dokunup 175 milisaniye basılı tuttuğunuzda, etrafındaki aynı değerli ve birbirine bağlı tüm taşlar parlar. Parmağınızı kaldırdığınızda hepsi birleşir.

Bu mekanizma 3 aşamadan oluşur:

1. **Dokunma algılama:** Hangi taşa basıldı?
2. **BFS arama:** Bağlı eşdeğer taşları bul ve vurgula.
3. **Merge:** Parmak kaldırılınca taşları birleştir.

6.2 Dokunma Girdisini Almak

Crystal.gd'e dokunma olaylarını ekleyin:

```
# Crystal.gd'e eklenecek — dokunma sinyalleri

signal crystal_pressed(row, col)
signal crystal_released(row, col)

# Bu fonksiyon dokunma/tıklama olaylarını yakalar
func _gui_input(event: InputEvent):
    if event is InputEventScreenTouch or event is InputEventMouseButton:
        if event.pressed:
            emit_signal("crystal_pressed", grid_row, grid_col)
        else:
            emit_signal("crystal_released", grid_row, grid_col)
```

📌 **NOT:** InputEventScreenTouch mobil cihazlar için, InputEventMouseButton ise masaüstü için çalışır. İkisini birden kontrol etmek hem bilgisayarda hem telefonda test etmenizi sağlar.

6.3 GameManager'da Hold Yönetimi

```
# GameManager.gd'e eklenecek değişkenler:
var hold_row: int = -1
var hold_col: int = -1
var hold_matches: Array[Vector2i] = []  # Bulunan eşleşen hücreler
var is_holding: bool = false
var hold_elapsed: float = 0.0

# create_block() içinde sinyal bağlama ekle:
func create_block(row: int, col: int, value: int) -> Node:
    # ... (önceki kod)
    crystal.connect("crystal_pressed", _on_crystal_pressed)
    crystal.connect("crystal_released", _on_crystal_released)
    return crystal
```

```
# Taşa basıldığında
func _on_crystal_pressed(row: int, col: int):
    if not is_active or is_busy: return
    if G[row][col] == 0: return

    hold_row = row
    hold_col = col
    hold_elapsed = 0.0
    is_holding = true

# Taştan el kaldırıldığında
func _on_crystal_released(row: int, col: int):
    if not is_active or is_busy: return
    is_holding = false

    if hold_matches.size() > 0:
        do_hold_merge()
    else:
        cancel_hold()

# Her karede hold süresini say
func _process(delta: float):
    if is_holding and hold_row >= 0:
        hold_elapsed += delta
        if hold_elapsed >= 0.175 and hold_matches.size() == 0:
            begin_hold_search()

# Hold iptal et
func cancel_hold():
    # Tüm "parlaklığı" kaldır
    for cell in hold_matches:
        var key = "%d-%d" % [cell.x, cell.y]
        if block_elements.has(key):
            block_elements[key].set_highlighted(false)
    # Merkez taşı da
    if hold_row >= 0:
        var key = "%d-%d" % [hold_row, hold_col]
        if block_elements.has(key):
            block_elements[key].set_selected(false)

    hold_row = -1
    hold_col = -1
    hold_matches = []
    hold_elapsed = 0.0
```

6.4 BFS Algoritması — Eşleşenleri Bul

BFS (Breadth-First Search — Genişlik Öncelikli Arama), bir ızgarada birbirine bağlı aynı değerli taşları bulmak için kullanılır. Şöyle çalışır:

4. Başlangıç hücresini bir kuyruğa (queue) ekle.
5. Kuyruğun önündeki hücreyi al, 8 komşusuna bak.
6. Komşu aynı değere sahipse ve henüz ziyaret edilmediyse kuyruğa ekle.
7. Kuyruk boşalana kadar devam et.

```
# BFS ile bağlı eşdeğer taşları bul
func begin_hold_search():
    var v = G[hold_row][hold_col]
    if v == 0: return

    hold_matches = []
    var visited: Dictionary = {}
```

```
    var queue: Array = [Vector2i(hold_row, hold_col)]
    visited["%d-%d" % [hold_row, hold_col]] = true

    while queue.size() > 0:
        var current = queue.pop_front()   # Kuyruğun önünden al
        var cr = current.x
        var cc = current.y

        # 8 yönlü komşuları kontrol et
        for dr in [-1, 0, 1]:
            for dc in [-1, 0, 1]:
                if dr == 0 and dc == 0: continue  # Kendisi değil
                var nr = cr + dr
                var nc = cc + dc
                var nkey = "%d-%d" % [nr, nc]

                if is_valid_cell(nr, nc) and not visited.has(nkey):
                    if G[nr][nc] == v:   # Aynı değer mi?
                        visited[nkey] = true
                        hold_matches.append(Vector2i(nr, nc))
                        queue.append(Vector2i(nr, nc))

    # Görsel geri bildirim: eşleşenleri parla
    _highlight_matches()

func _highlight_matches():
    # Merkez taşı "seçili" göster
    var ckey = "%d-%d" % [hold_row, hold_col]
    if block_elements.has(ckey):
        block_elements[ckey].set_selected(true)

    # Eşleşenleri "vurgu" rengiyle göster
    for cell in hold_matches:
        var key = "%d-%d" % [cell.x, cell.y]
        if block_elements.has(key):
            block_elements[key].set_highlighted(true)
```

6.5 Crystal.gd'e Görsel Durum Fonksiyonları Ekleme

```
# Crystal.gd'e eklenecek fonksiyonlar

var is_selected: bool = false
var is_highlighted: bool = false

func set_selected(val: bool):
    is_selected = val
    if val:
        # Beyaz parlak kenarlık göster, biraz küçült
        scale = Vector2(0.9, 0.9)
        # Parlak beyaz glow efekti ekle
        _apply_glow_style(Color.WHITE, 4)
    else:
        scale = Vector2.ONE
        update_visuals()

func set_highlighted(val: bool):
    is_highlighted = val
    if val:
        # Titreşen animasyon (Tween ile)
        _start_pulse_animation()
    else:
        _stop_pulse_animation()
        update_visuals()

func _apply_glow_style(glow_color: Color, border_width: int):
```

```
    var color = get_color_for_value(cell_value)
    var style = create_stylebox(color)
    style.border_color = glow_color
    style.border_width_left = border_width
    style.border_width_right = border_width
    style.border_width_top = border_width
    style.border_width_bottom = border_width
    add_theme_stylebox_override("panel", style)

var pulse_tween: Tween = null

func _start_pulse_animation():
    if pulse_tween: pulse_tween.kill()
    pulse_tween = create_tween().set_loops()
    pulse_tween.tween_property(self, "scale", Vector2(1.06, 1.06), 0.2)
    pulse_tween.tween_property(self, "scale", Vector2(1.0, 1.0), 0.2)

func _stop_pulse_animation():
    if pulse_tween: pulse_tween.kill()
    pulse_tween = null
    scale = Vector2.ONE
```

BÖLÜM 7

Merge Sistemi

Taşları Birleştir

7.1 Merge Formülü

Merge gerçekleştiğinde yeni değer şu formüle göre hesaplanır:

```
Yeni Değer = Temel Değer × 2^ceil(N/2)
```

N = Toplam taş sayısı (merkez dahil, eşleşenler + 1)

Durum	Hesaplama
2 taş (N=2)	4: 2^ceil(2/2) = 2^1 = ×2 → örn: 32 + 32 = 64
3 taş (N=3)	4: 2^ceil(3/2) = 2^2 = ×4 → örn: 32 × 4 = 128
5 taş (N=5)	4: 2^ceil(5/2) = 2^3 = ×8 → örn: 32 × 8 = 256
13 taş (N=13)	4: 2^ceil(13/2) = 2^7 = ×128

7.2 Puan Sistemi

Her merge belirli bir puan verir. Puan hesabı:

```
# log2(yeni_deger) × toplam_tas × combo_katsayi

# Combo katsayısı:
#   3-4 taş → ×1.2
#   5-7 taş → ×1.5
#   8+ taş  → ×2.0 (🔥 MEGA COMBO!)
```

7.3 do_hold_merge() Fonksiyonu

```
# GameManager.gd'e eklenecek

func do_hold_merge():
    if hold_row < 0 or hold_matches.size() == 0: return

    var center_r = hold_row
    var center_c = hold_col
    var base_val = G[center_r][center_c]
    var total_tiles = hold_matches.size() + 1  # Eşleşen + merkez
```

```
    # Yeni değer hesapla: baseV × 2^ceil(N/2)
    var power = ceili(total_tiles / 2.0)
    var new_value = base_val * int(pow(2, power))

    # Geçmişe kaydet (geri alma için)
    save_history()

    is_busy = true

    # 1. Eşleşen taşları merkeze uçur ve sil
    var center_pos = block_elements["%d-%d" % [center_r,
center_c]].global_position

    for cell in hold_matches:
        var key = "%d-%d" % [cell.x, cell.y]
        var el = block_elements.get(key)
        if not el: continue

        # G matrisinden anında sil (gravity bunu boş sayar)
        G[cell.x][cell.y] = 0
        block_elements.erase(key)

        # Merkeze doğru uçur animasyonu
        var tween = create_tween()
        tween.tween_property(el, "global_position", center_pos, 0.22)
        tween.parallel().tween_property(el, "scale", Vector2(0.2, 0.2), 0.22)
        tween.parallel().tween_property(el, "modulate:a", 0.0, 0.22)
        # Animasyon bitince node'u sil
        tween.tween_callback(el.queue_free)

    # 2. 250ms bekle, sonra merkez taşı güncelle
    await get_tree().create_timer(0.25).timeout

    G[center_r][center_c] = new_value
    var center_key = "%d-%d" % [center_r, center_c]
    var center_el = block_elements.get(center_key)
    if center_el:
        center_el.cell_value = new_value
        center_el.set_selected(false)
        center_el.update_visuals()
        # Büyüme animasyonu
        var merge_tween = create_tween()
        merge_tween.tween_property(center_el, "scale", Vector2(1.4, 1.4), 0.1)
        merge_tween.tween_property(center_el, "scale", Vector2(1.0, 1.0), 0.15)

    # 3. Puanla
    var count = hold_matches.size()
    var combo = 2.0 if count >= 8 else (1.5 if count >= 5 else (1.2 if count >= 3
else 1.0))
    var log_v = log(new_value) / log(2)
    var pts = int(log_v * total_tiles * combo)
    score += pts
    coins += 1
    last_merge_count = total_tiles

    # 4. Hold durumunu sıfırla
    hold_row = -1
    hold_col = -1
    hold_matches = []

    # 5. Hedef kontrolü
    if new_value >= TARGETS[target_idx]:
        await level_complete()
        return

    # 6. Yerçekimi ve spawn
    await get_tree().create_timer(0.05).timeout
    apply_gravity()
    await get_tree().create_timer(0.25).timeout
```

```
    spawn_tiles()
    await get_tree().create_timer(0.35).timeout

    update_hud()
    is_busy = false
```

⚠️ **DİKKAT:** "await" komutu çok önemlidir. Bir sonraki satıra geçmeden önce belirtilen sürenin dolmasını bekler. Bu olmadan animasyonlar üst üste biner ve oyun bozulur.

BÖLÜM 8

Yerçekimi

Taşlar Düşsün

8.1 Yerçekimi Nasıl Çalışır?

Merge sonrasında oluşan boşluklar, üstteki taşların aşağı kaymasıyla dolar. Bu "yerçekimi" efektidir.

Algoritma: Her sütun ayrı ayrı işlenir. Aşağıdan yukarıya doğru taranır. Boş olmayan taşlar bir sonraki boş konuma yazılır.

Örnek — sütun 2:

```
Merge öncesi:   Merge sonrası (0=boş):   Yerçekimi sonrası:
[16]            [16]                      [ 0]
[32]            [ 0]  ← silindi           [16]
[32]            [ 0]  ← silindi           [64]
[64]            [64]                      [ 8]
[ 8]            [ 8]                      [ 4]
[ 4]            [ 4]                      [ 2]
[128]           [128]                     [128]
```

8.2 apply_gravity() Fonksiyonu

```
# GameManager.gd'e eklenecek

func apply_gravity():
    var any_moved = false

    for col in range(COLS):
        # write_row: yazılacak satır, alttan başla
        var write_row = ROWS - 1

        # Alttan yukarıya tara
        for row in range(ROWS - 1, -1, -1):
            if G[row][col] != 0:
                if row != write_row:
                    # Taşı aşağı taşı
                    G[write_row][col] = G[row][col]
                    G[row][col] = 0

                    # DOM'u güncelle
                    move_block(row, col, write_row, col)
                    any_moved = true
                write_row -= 1

    return any_moved

# Bir taşı animasyonla farklı konuma taşı
func move_block(from_r: int, from_c: int, to_r: int, to_c: int):
    var old_key = "%d-%d" % [from_r, from_c]
    var new_key = "%d-%d" % [to_r, to_c]

    if not block_elements.has(old_key): return
```

```
    var el = block_elements[old_key]

    # Referans haritasını güncelle
    block_elements.erase(old_key)
    block_elements[new_key] = el

    # Konumunu güncelle
    el.grid_row = to_r
    el.grid_col = to_c

    # Animasyonlu kayma
    var target_pos = cell_to_pos(to_r, to_c)
    var tween = create_tween()
    tween.set_ease(Tween.EASE_IN_OUT)
    tween.set_trans(Tween.TRANS_CUBIC)
    tween.tween_property(el, "position", target_pos, 0.18)
```

💡 **İPUCU:** apply_gravity() senkron çalışır (await yok). Animasyonlar paralel başlar. Çağrıldıktan sonra 0.2 saniye bekleyin ki animasyonlar tamamlansın.

BÖLÜM 9

Spawn Sistemi

Yeni Taşlar Üret

9.1 Spawn Mantığı

Merge ve yerçekimi sonrasında oluşan boş hücreler yeni taşlarla dolar. Hangi değerin geleceği 3 katmanlı ağırlıklı bir sisteme göre belirlenir:

Katman	Açıklama
Katman 1 — BASE (%65)	2, 4, 8, 16 → Her zaman dominant. Bunlar "oyunun oksijeni"
Katman 2 — KÖPRÜ (%10)	Orta değerler. BASE ile MID arasındaki geçiş.
Katman 3 — MID (%9)	Hedefe bağlı orta değer. Bölüm ilerledikçe artar.
Katman 4 — HIGH (%5)	Nadir yüksek değer. "Ödül hissi" verir.
Kurtarıcı spawn (%30)	Gridde mevcut olan değerlerden biri — merge fırsatı yaratır.
Doluluk koruması	Grid %75+ doluysa sadece 2, 4, 8 gelir — tıkanmayı önler.

9.2 random_spawn_value() Fonksiyonu

```
func random_spawn_value() -> int:
    var target = TARGETS[target_idx]

    # Doluluk koruması: %75+ doluysa küçük değer
    var filled = 0
    for r in range(ROWS):
        for c in range(COLS):
            if G[r][c] != 0: filled += 1
    if float(filled) / float(ROWS * COLS) >= 0.75:
        return [2, 2, 4, 4, 8][randi() % 5]

    # Kurtarıcı spawn: %30 ihtimalle gridde var olan bir değer
    if randf() < 0.30:
        var existing: Array[int] = []
        for r in range(ROWS):
            for c in range(COLS):
                if G[r][c] != 0: existing.append(G[r][c])
        if existing.size() > 0:
            # Frekans say
            var freq: Dictionary = {}
            for v in existing:
                freq[v] = freq.get(v, 0) + 1
            # En sık tekrar eden 3 değerden seç
            var sorted_keys = freq.keys()
            sorted_keys.sort_custom(func(a, b): return freq[b] < freq[a])
```

```
            var top3 = sorted_keys.slice(0, min(3, sorted_keys.size()))
            return top3[randi() % top3.size()]

    # Ağırlıklı havuz sistemi
    var snap = func(v): return int(pow(2, round(log(v) / log(2))))
    var mid = snap.call(max(8, target / 12))
    var high = snap.call(max(16, target / 6))
    var mid_low = snap.call(max(32, mid / 4))

    # Havuz: her değer ağırlığı kadar tekrar eklendi
    var pool: Array[int] = []
    for i in 38: pool.append(2)
    for i in 28: pool.append(4)
    for i in 16: pool.append(8)
    for i in 9:  pool.append(16)
    for i in 14: pool.append(mid_low)
    for i in 12: pool.append(mid)
    for i in 7:  pool.append(high)

    return pool[randi() % pool.size()]
```

9.3 spawn_tiles() Fonksiyonu

```
func spawn_tiles():
    # Freeze aktifse bu turu atla
    if freeze_active:
        freeze_active = false
        update_hud()
        return

    var spawn_count = 0

    # Her sütunu kontrol et
    for col in range(COLS):
        for row in range(ROWS):
            if G[row][col] == 0:
                var val = random_spawn_value()
                G[row][col] = val

                # Gecikmeli spawn animasyonu (cascade efekti)
                var delay = spawn_count * 0.04
                var r_capture = row
                var c_capture = col
                var v_capture = val

                get_tree().create_timer(delay).timeout.connect(
                    func(): _spawn_single(r_capture, c_capture, v_capture))
                spawn_count += 1

func _spawn_single(row: int, col: int, val: int):
    var crystal = create_block(row, col, val)
    if not crystal: return

    # Yukarıdan düşme animasyonu
    var start_pos = cell_to_pos(row, col) + Vector2(0, -80)
    var end_pos = cell_to_pos(row, col)

    crystal.position = start_pos
    crystal.modulate.a = 0.0

    var tween = create_tween()
    tween.set_ease(Tween.EASE_OUT)
    tween.set_trans(Tween.TRANS_BOUNCE)
    tween.tween_property(crystal, "position", end_pos, 0.22)
    tween.parallel().tween_property(crystal, "modulate:a", 1.0, 0.1)
```

9.4 İlk Grid Dolumu

Oyun başlangıcında tüm 35 hücreyi doldur. init_grid() içine ekle:

```
func init_grid():
    G = []
    # Boş 7×5 matris oluştur
    for r in range(ROWS):
        var row = []
        for c in range(COLS):
            row.append(0)
        G.append(row)

    # Tüm hücreleri spawn et
    spawn_tiles()
```

BÖLÜM 10

Puan, Coin ve HUD Arayüzü

Bilgi Ekranı Tasarımı

10.1 HUD Sahnesi Oluşturma

HUD (Heads-Up Display), oyun sırasında sürekli görünen bilgi ekranıdır. CanvasLayer içine koyuyoruz ki grid animasyonlarından etkilenmesin.

```
HUD (CanvasLayer)
├── ScorePanel (PanelContainer)
│   ├── VBox (VBoxContainer)
│   │   ├── ScoreLabel (Label)      ← "PUAN"
│   │   └── ScoreValue (Label)      ← "0"
│   └── CoinRow (HBoxContainer)
│       ├── CoinIcon (Label)        ← "🪙"
│       └── CoinValue (Label)       ← "30"
├── TargetPanel (PanelContainer)
│   ├── TargetLabel (Label)         ← "HEDEF"
│   └── TargetValue (Label)         ← "2048"
└── PowerupPanel (HBoxContainer)
    ├── SwapBtn (Button)
    ├── UndoBtn (Button)
    ├── SmashBtn (Button)
    ├── BombBtn (Button)
    ├── FreezeBtn (Button)
    └── ShuffleBtn (Button)
```

10.2 HUD.gd Script

```
extends CanvasLayer

@onready var score_value: Label = $ScorePanel/VBox/ScoreValue
@onready var coin_value: Label = $ScorePanel/CoinRow/CoinValue
@onready var target_value: Label = $TargetPanel/TargetValue

# HUD'u güncelle
func update(score: int, coins: int, target: int):
    score_value.text = str(score)
    coin_value.text = str(coins)
    target_value.text = format_value(target)

    # Puan pop animasyonu
    _pop_animation(score_value)

func _pop_animation(label: Label):
    var tween = create_tween()
    tween.tween_property(label, "scale", Vector2(1.35, 1.35), 0.1)
    tween.tween_property(label, "scale", Vector2(1.0, 1.0), 0.15)

# Büyük sayıları kısa formata çevir
func format_value(v: int) -> String:
    if v >= 1073741824: return str(v / 1073741824) + "G"
    if v >= 1048576:    return str(v / 1048576) + "M"
    if v >= 1024:       return str(v / 1024) + "K"
```

```
    return str(v)

# Uçan puan efekti (+123 yazısı yukarı uçar)
func show_floating_score(value: int, world_pos: Vector2):
    var lbl = Label.new()
    lbl.text = "+" + str(value)
    lbl.add_theme_color_override("font_color", Color("#fbbf24"))
    lbl.add_theme_font_size_override("font_size", 20)
    add_child(lbl)
    lbl.global_position = world_pos

    var tween = create_tween()
    tween.tween_property(lbl, "position:y", lbl.position.y - 60, 0.9)
    tween.parallel().tween_property(lbl, "modulate:a", 0.0, 0.9)
    tween.tween_callback(lbl.queue_free)
```

10.3 GameManager'da HUD Güncelleme

```
# GameManager.gd'de HUD referansı
@onready var hud = $HUD

func update_hud():
    hud.update(score, coins, TARGETS[target_idx])

# Power-up butonlarını güncelle (yeterli coin var mı?)
func update_powerup_buttons():
    var btns = {
        "swap": $HUD/PowerupPanel/SwapBtn,
        "undo": $HUD/PowerupPanel/UndoBtn,
        "smash": $HUD/PowerupPanel/SmashBtn,
        "bomb": $HUD/PowerupPanel/BombBtn,
        "freeze": $HUD/PowerupPanel/FreezeBtn,
        "shuffle": $HUD/PowerupPanel/ShuffleBtn,
    }
    for id in btns:
        var btn = btns[id]
        var cost = PU_COST[id]
        btn.disabled = coins < cost
        btn.text = "%s\n🪙%d" % [btn.name, cost]
```

BÖLÜM 11

Power-up Sistemi

Özel Güçler

11.1 Power-up Sistemi Genel Bakış

6 farklı power-up var. Hepsini coin ile satın alırsınız. Her birinin farklı bir modu vardır:

Power-up	Nasıl Çalışır?
🔄 **Swap (40 coin)**	İki komşu taşın yerini değiştirir. İki taşa sırayla tıkla.
↩️ **Undo (50 coin)**	Son hamleyi geri alır. G matrisinin anlık snapshot'ı kullanılır.
🔨 **Smash (50 coin)**	Tek bir taşı siler. Tıkladığın taş patlar.
💣 **Bomb (120 coin)**	3×3 alandaki tüm taşları temizler. Tıkladığın yerin çevresini patlatır.
❄️ **Freeze (60 coin)**	Bir sonraki spawn turunu atlar. Hemen aktif olur.
🔀 **Shuffle (80 coin)**	Tüm taşları rastgele yeniden dağıtır.

11.2 Power-up Mod Sistemi

```
# GameManager.gd'e eklenecek
var current_powerup: String = ""   # Aktif mod: "smash", "bomb", "swap", ""
var swap_first: Vector2i = Vector2i(-1, -1)   # Swap'ta ilk seçilen taş

# Power-up satın al ve modu aktifleştir
func activate_powerup(id: String):
    if not is_active or is_busy: return
    var cost = PU_COST[id]
    if coins < cost: return

    # Freeze anında çalışır, mod gerektirmez
    if id == "freeze":
        coins -= cost
        freeze_active = true
        update_hud()
        return

    # Shuffle anında çalışır
    if id == "shuffle":
        coins -= cost
        do_shuffle()
        update_hud()
        return

    # Undo anında çalışır
    if id == "undo":
        coins -= cost
        do_undo()
```

```
        update_hud()
        return

    # Diğerleri mod gerektiriyor
    coins -= cost
    current_powerup = id
    swap_first = Vector2i(-1, -1)
    update_hud()
    # İmleç değiştir (opsiyonel)

# Crystal tıklandığında power-up mod kontrolü
func _on_crystal_pressed(row: int, col: int):
    if not is_active or is_busy: return

    match current_powerup:
        "smash": do_smash(row, col); return
        "bomb":  do_bomb(row, col);  return
        "swap":  do_swap(row, col);  return

    # Normal hold mekanizması
    hold_row = row
    hold_col = col
    hold_elapsed = 0.0
    is_holding = true
```

11.3 Smash — Tek Taş Patlat

```
func do_smash(row: int, col: int):
    if G[row][col] == 0: return

    save_history()
    is_busy = true
    current_powerup = ""

    # Taşı görsel efektle sil
    var key = "%d-%d" % [row, col]
    var el = block_elements.get(key)
    if el:
        block_elements.erase(key)
        G[row][col] = 0

        # Patlama animasyonu
        var tween = create_tween()
        tween.tween_property(el, "scale", Vector2(1.5, 1.5), 0.08)
        tween.parallel().tween_property(el, "modulate:a", 0.0, 0.2)
        tween.tween_callback(el.queue_free)

    await get_tree().create_timer(0.25).timeout
    apply_gravity()
    await get_tree().create_timer(0.22).timeout
    spawn_tiles()
    await get_tree().create_timer(0.3).timeout
    update_hud()
    is_busy = false
```

11.4 Bomb — 3×3 Alan Temizle

```
func do_bomb(row: int, col: int):
    save_history()
    is_busy = true
    current_powerup = ""
```

```
    # 3×3 etki alanı (merkez = tıklanan hücre)
    for dr in [-1, 0, 1]:
        for dc in [-1, 0, 1]:
            var nr = row + dr
            var nc = col + dc
            if not is_valid_cell(nr, nc): continue
            if G[nr][nc] == 0: continue

            var key = "%d-%d" % [nr, nc]
            var el = block_elements.get(key)
            if el:
                block_elements.erase(key)
                G[nr][nc] = 0

                # Gecikme ile patlama (cascade efekti)
                var d = (abs(dr) + abs(dc)) * 0.05
                var el_cap = el
                get_tree().create_timer(d).timeout.connect(func():
                    var t = create_tween()
                    t.tween_property(el_cap, "scale", Vector2(1.8, 1.8), 0.1)
                    t.parallel().tween_property(el_cap, "modulate",
Color(1,0.5,0,0), 0.2)
                    t.tween_callback(el_cap.queue_free)
                )

    await get_tree().create_timer(0.35).timeout
    apply_gravity()
    await get_tree().create_timer(0.22).timeout
    spawn_tiles()
    await get_tree().create_timer(0.3).timeout
    update_hud()
    is_busy = false
```

11.5 Swap — İki Taşın Yerini Değiştir

```
func do_swap(row: int, col: int):
    if G[row][col] == 0: return

    if swap_first == Vector2i(-1, -1):
        # İlk taş seçildi
        swap_first = Vector2i(row, col)
        var key = "%d-%d" % [row, col]
        if block_elements.has(key):
            block_elements[key].set_selected(true)
        return

    # İkinci taş seçildi — yer değiştir
    var r1 = swap_first.x; var c1 = swap_first.y
    var r2 = row; var c2 = col

    # Komşu mu? (8 yön)
    if abs(r1-r2) > 1 or abs(c1-c2) > 1:
        # Komşu değil — seçimi sıfırla
        var key1 = "%d-%d" % [r1, c1]
        if block_elements.has(key1): block_elements[key1].set_selected(false)
        swap_first = Vector2i(-1, -1)
        return

    save_history()
    current_powerup = ""

    # G matrisinde değerleri değiştir
    var temp = G[r1][c1]
    G[r1][c1] = G[r2][c2]
```

```
    G[r2][c2] = temp

    # Görsel — animasyonlu yer değiştirme
    var key1 = "%d-%d" % [r1, c1]
    var key2 = "%d-%d" % [r2, c2]
    var el1 = block_elements.get(key1)
    var el2 = block_elements.get(key2)

    if el1 and el2:
        el1.set_selected(false)
        block_elements.erase(key1); block_elements.erase(key2)
        block_elements[key2] = el1; block_elements[key1] = el2
        el1.grid_row = r2; el1.grid_col = c2
        el2.grid_row = r1; el2.grid_col = c1

        var t1 = create_tween()
        t1.tween_property(el1, "position", cell_to_pos(r2, c2), 0.2)
        var t2 = create_tween()
        t2.tween_property(el2, "position", cell_to_pos(r1, c1), 0.2)

    swap_first = Vector2i(-1, -1)
    update_hud()
```

11.6 Undo — Geri Al

```
# Anlık durum snapshot'ı al
func save_history():
    var snapshot = {
        "G": [],
        "score": score,
        "coins": coins
    }
    # G matrisinin derin kopyasını al
    for r in range(ROWS):
        var row = []
        for c in range(COLS):
            row.append(G[r][c])
        snapshot["G"].append(row)

    history.append(snapshot)
    if history.size() > 5:   # Sadece son 5 hamle sakla
        history.pop_front()

func do_undo():
    if history.size() == 0: return
    var snap = history.pop_back()

    score = snap["score"]
    coins = snap["coins"]
    G = snap["G"]

    rebuild_dom()   # Tüm görsel taşları yeniden çiz
    update_hud()

# Tüm görsel taşları G matrisinden yeniden oluştur
func rebuild_dom():
    # Eski taşları sil
    for key in block_elements:
        if block_elements[key]:
            block_elements[key].queue_free()
    block_elements.clear()

    # G matrisinden yeniden oluştur
    for r in range(ROWS):
        for c in range(COLS):
```

```
            if G[r][c] != 0:
                create_block(r, c, G[r][c])
```

11.7 Shuffle — Tüm Taşları Karıştır

```
func do_shuffle():
    save_history()

    # Gridde bulunan tüm değerleri topla
    var values: Array[int] = []
    for r in range(ROWS):
        for c in range(COLS):
            if G[r][c] != 0:
                values.append(G[r][c])

    # Fisher-Yates algoritması ile karıştır
    for i in range(values.size() - 1, 0, -1):
        var j = randi() % (i + 1)
        var tmp = values[i]
        values[i] = values[j]
        values[j] = tmp

    # G matrisini güncelle
    var idx = 0
    for r in range(ROWS):
        for c in range(COLS):
            if G[r][c] != 0:
                G[r][c] = values[idx]
                idx += 1

    rebuild_dom()
```

BÖLÜM 12

Ses Efektleri

AudioStreamPlayer ile Dinamik Ses

12.1 Godot'ta Ses Sistemi

Godot'ta ses çalmak için AudioStreamPlayer node'u kullanılır. İki yolu var:

- **Ses dosyası:** .ogg veya .wav dosyası içe aktarıp çalabilirsiniz.
- **Dinamik ses:** AudioStreamGenerator ile kod ile ses üretebilirsiniz (Web Audio API karşılığı).

Basit yaklaşım: Ses dosyası kullanın. Ücretsiz ses efektleri için https://freesound.org adresini kullanabilirsiniz.

> 💡 **İPUCU:** Oyununuz için en az şu sesler gerekli: merge sesi, spawn sesi, buton tıklama sesi, kazanma fanfaresi, game over sesi. Bunları .ogg formatında indirin.

12.2 Ses Node Yapısı

```
Main (Node2D)
└── SoundManager (Node)
    ├── MergeSound (AudioStreamPlayer)
    ├── SpawnSound (AudioStreamPlayer)
    ├── ButtonSound (AudioStreamPlayer)
    ├── WinSound (AudioStreamPlayer)
    └── GameOverSound (AudioStreamPlayer)
```

Her AudioStreamPlayer için Inspector'dan Stream alanına .ogg/.wav dosyasını atayın.

12.3 SoundManager.gd

```
extends Node

var sound_on: bool = true

@onready var merge_sound   = $MergeSound
@onready var spawn_sound   = $SpawnSound
@onready var button_sound  = $ButtonSound
@onready var win_sound     = $WinSound
@onready var gameover_sound = $GameOverSound

func play_merge(value: int):
    if not sound_on: return
    # Değere göre pitch (ton yüksekliği) ayarla
    var pitch = 0.8 + (log(value) / log(2)) * 0.05
    merge_sound.pitch_scale = clamp(pitch, 0.5, 2.0)
    merge_sound.play()
```

```
func play_spawn():
    if not sound_on: return
    spawn_sound.pitch_scale = randf_range(0.9, 1.1)
    spawn_sound.play()

func play_button():
    if not sound_on: return
    button_sound.play()

func play_win():
    if not sound_on: return
    win_sound.play()

func play_gameover():
    if not sound_on: return
    gameover_sound.play()

func toggle_sound():
    sound_on = !sound_on
    # Tüm player'ların volume_db'sini ayarla
    var vol = 0.0 if sound_on else -80.0
    for child in get_children():
        if child is AudioStreamPlayer:
            child.volume_db = vol
```

📌 **NOT:** pitch_scale özelliği sesin tonunu değiştirir. 1.0 = normal, 2.0 = iki kat tiz, 0.5 = iki kat pes. Merge değerine göre değiştirerek farklı sesler elde edebilirsiniz.

BÖLÜM 13

Kayıt ve Yükleme Sistemi

Oyun Durumunu Sakla

13.1 Kayıt Sistemi Neden Gerekli?

Oyuncu uygulamayı kapattığında oyun durumu kaybolmamalıdır. Godot'ta kayıt için ConfigFile veya JSON kullanabiliriz.

13.2 Kaydedilecek Veriler

Değişken	Neden Kaydedilir?
G (grid matrisi)	Tüm taşların değerleri — 2D dizi
score	Toplam puan
coins	Mevcut coin miktarı
target_idx	Hangi bölümde olunduğu
freeze_active	Freeze aktif mi?

13.3 save_game() ve load_game()

```
# GameManager.gd'e eklenecek

const SAVE_PATH = "user://axydrop_save.cfg"

func save_game():
    if not is_active: return

    var config = ConfigFile.new()
    config.set_value("state", "score", score)
    config.set_value("state", "coins", coins)
    config.set_value("state", "target_idx", target_idx)
    config.set_value("state", "freeze_active", freeze_active)

    # G matrisini düz diziye çevir (ConfigFile ile saklamak için)
    var flat_g: Array[int] = []
    for r in range(ROWS):
        for c in range(COLS):
            flat_g.append(G[r][c])
    config.set_value("state", "grid", flat_g)

    config.save(SAVE_PATH)

func load_game() -> bool:
    var config = ConfigFile.new()
    if config.load(SAVE_PATH) != OK:
        return false   # Kayıt yok
```

```
    score = config.get_value("state", "score", 0)
    coins = config.get_value("state", "coins", 30)
    target_idx = config.get_value("state", "target_idx", 0)
    freeze_active = config.get_value("state", "freeze_active", false)

    var flat_g = config.get_value("state", "grid", [])
    if flat_g.size() == ROWS * COLS:
        G = []
        for r in range(ROWS):
            var row = []
            for c in range(COLS):
                row.append(flat_g[r * COLS + c])
            G.append(row)
        rebuild_dom()

    update_hud()
    return true

# Oyun kapanırken veya arka plana geçince kaydet
func _notification(what: int):
    if what == NOTIFICATION_WM_GO_BACK_REQUEST or \
       what == NOTIFICATION_APPLICATION_PAUSED:
        save_game()

# Her 3 saniyede otomatik kaydet
var auto_save_timer: float = 0.0

func _process(delta: float):
    auto_save_timer += delta
    if auto_save_timer >= 3.0 and is_active and not is_busy:
        auto_save_timer = 0.0
        save_game()
```

13.4 Oyun Başlangıcında Yükleme

```
func _ready():
    # Kayıt var mı kontrol et
    if load_game():
        # Kaldığı yerden devam et
        is_active = true
        update_hud()
    else:
        # Yeni oyun başlat
        init_grid()
        is_active = true
```

BÖLÜM 14

Son Dokunuşlar ve Yayınlama

Bitirme ve Dışa Aktarma

14.1 Game Over Kontrolü

Oyun, grid tamamen dolduğunda ve hiçbir eşleşen komşu kalmadığında biter.

```
func check_game_over() -> bool:
    # Grid'de boş hücre var mı?
    for r in range(ROWS):
        for c in range(COLS):
            if G[r][c] == 0: return false  # Boş hücre var, devam et

    # Herhangi bir komşu eşleşme var mı?
    for r in range(ROWS):
        for c in range(COLS):
            for dr in [-1, 0, 1]:
                for dc in [-1, 0, 1]:
                    if dr == 0 and dc == 0: continue
                    var nr = r + dr; var nc = c + dc
                    if is_valid_cell(nr, nc) and G[nr][nc] == G[r][c]:
                        return false  # Eşleşme var, devam et

    return true  # Ne boş hücre ne eşleşme — GAME OVER

# Merge sonrası game over kontrolü (spawn'dan sonra çağır)
func after_spawn_checks():
    if check_game_over():
        is_active = false
        $GameOverScreen.show()
        sound_manager.play_gameover()
```

14.2 Bölüm Tamamlama

```
func level_complete():
    is_active = false
    sound_manager.play_win()

    # Kutlama efekti
    _show_confetti()

    await get_tree().create_timer(2.0).timeout

    # Sonraki bölüme geç
    target_idx += 1
    if target_idx >= TARGETS.size():
        target_idx = TARGETS.size() - 1  # Son bölümde kal

    # Coin ödülü
    coins += 20

    is_active = true
    update_hud()

func _show_confetti():
```

```
    # Ekranda parçacık efekti (GPUParticles2D ile)
    var particles = $ConfettiParticles
    particles.emitting = true
    await get_tree().create_timer(1.5).timeout
    particles.emitting = false
```

14.3 Mobil İçin Dışa Aktarma — Android

Gereksinimler: Android Studio ve Android SDK kurulu olmalı. Godot'ta Export menüsünden ayarlayabilirsiniz.

8. Project → Export → "Add..." → "Android" seçin
9. Godot'un Android SDK yolunu otomatik bulmasını bekleyin ya da kendiniz gösterin
10. Keystore oluşturun (Play Store'a yüklemek için imzalı APK gerekir)
11. "Export Project" butonuyla .apk oluşturun
12. Android telefonunuza "Bilinmeyen Kaynaklardan Yükleme" açık olmalı

14.4 Mobil İçin Dışa Aktarma — iOS

Gereksinimler: macOS ve Xcode kurulu olmalı.

13. Project → Export → "Add..." → "iOS" seçin
14. Xcode proje dosyası oluşturulur
15. Xcode'da açın → gerçek cihaza yükleyin ya da App Store Connect'e gönderin

14.5 Web (HTML5) İçin Dışa Aktarma

16. Project → Export → "Add..." → "Web" seçin
17. "Export Project" ile .html + .wasm + .pck dosyaları oluşur
18. Bu dosyaları bir web sunucusuna yükleyin

> 💡 **İPUCU:** Itch.io veya GitHub Pages'e ücretsiz yayınlayabilirsiniz. Web sürümünü kolayca paylaşabilirsiniz.

14.6 Performans İpuçları

- **Tween kullan:** AnimationPlayer yerine Tween daha hafiftir ve kodla yönetmek daha kolaydır.
- **queue_free() kullan:** Node silmek için free() yerine queue_free() kullanın — o frame'in sonunda güvenle silinir.
- **G matrisini kontrol et:** Her görsel güncelleme öncesi G matrisini kontrol edin. Ghost (hayalet) taş sorunları bu kurala uyarak önlenir.
- **await kullan:** Animasyonların tamamlanmasını await ile bekleyin. is_busy flag'ini doğru kullanın.
- **print() kaldır:** Yayın öncesi tüm print() satırlarını silin ya da yorum satırı yapın.

14.7 Projenin Tam Dosya Listesi

```
res://
├── project.godot            ← Proje ayarları (otomatik)
├── scenes/
│   ├── Main.tscn            ← Ana oyun sahnesi
│   ├── Crystal.tscn         ← Taş sahnesi
│   └── GameOver.tscn        ← Oyun bitti ekranı
├── scripts/
│   ├── GameManager.gd       ← Tüm oyun mantığı (~400 satır)
│   ├── Crystal.gd           ← Taş davranışı (~100 satır)
│   ├── HUD.gd               ← Arayüz güncelleme (~80 satır)
│   └── SoundManager.gd      ← Ses yönetimi (~60 satır)
├── audio/
│   ├── merge.ogg
│   ├── spawn.ogg
│   ├── button.ogg
│   ├── win.ogg
│   └── gameover.ogg
└── fonts/
    └── orbitron_bold.ttf    ← Oyun fontu (opsiyonel)
```

14.8 Sık Karşılaşılan Sorunlar ve Çözümleri

Sorun	Çözüm
Taşlar üst üste biniyor	G matrisini ve BE sözlüğünü senkron tutun. Her zaman önce G güncelleyin.
Animasyon çakışıyor	is_busy flag'ini doğru kullanın. Tüm işlemler is_busy=false iken yapılmalı.
Gravity yanlış çalışıyor	apply_gravity() için en az 200ms bekleyin. Animasyon süresinden önce çağırırsanız bozulur.
Spawn sonsuz döngüye giriyor	freeze_active kontrolünü spawn başında yapın. random_spawn_value() her zaman >0 döndürmeli.
Game Over erken geliyor	check_game_over() yalnızca spawn tamamlandıktan SONRA çağrılmalı.
Ses çalışmıyor	AudioStreamPlayer'a stream atandığından emin olun. Web'de kullanıcı etkileşimi olmadan ses çalmaz.
Export çalışmıyor	Export Templates indirilmeli: Editor → Manage Export Templates

> ✅ **TAMAMLANDI:** Tebrikler! Bu noktaya kadar geldiyseniz tam çalışan bir AXY Crystal Merge oyununuz var. Sıradaki adım: yeni özellikler eklemek, tasarımı geliştirmek ve yayınlamak!

🏆 SONUÇ — Buradan Nereye?

Bu kitapta sıfırdan tam bir mobil puzzle oyunu yapmayı öğrendiniz. Godot'un temel kavramlarını (Node, Scene, GDScript), oyun tasarımının temel unsurlarını (grid sistemi, merge mekaniği, yerçekimi, spawn, puan sistemi) ve profesyonel kodlama alışkanlıklarını (state yönetimi, animasyon zamanlaması, kayıt sistemi) öğrendiniz.

Sıradaki Adımlar

- Oyunun görsel tasarımını zenginleştirin: arka plan parçacıkları, ışıltı efektleri
- Yeni power-up'lar ekleyin: örneğin "Zaman Dondur" (5 saniye hamle yapma)
- Leaderboard (liderlik tablosu) ekleyin: Godot'un HTTP istekleri ile bir API'ye bağlanın
- Reklam entegrasyonu: AdMob ile gelir elde etmeye başlayın
- GameCenter / Google Play Games entegrasyonu: başarımlar ve skor tablosu
- Yeni oyun modları: zamana karşı mod, bulmaca modu

Faydalı Kaynaklar

- **Godot Dokümantasyonu:** https://docs.godotengine.org
- **GDScript Referansı:** https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/
- **Godot Öğrenme Merkezi:** https://godot.community/learning
- **Ücretsiz Ses Efektleri:** https://freesound.org
- **Ücretsiz Fontlar:** https://fonts.google.com (Orbitron önerilir)